

FurFeel - AI Architecture & Transition Roadmap

PART 1: Current Architecture

1. Posture & Motion Activity (Random Forest)

- The model determining the physical posture of the dog (e.g., walking, resting, shaking) is a Random Forest Classifier.
- Runs directly on the Arduino firmware (Collar) in real-time.
 - Uses 20Hz IMU data (Accelerometer and Gyroscope).
 - Data is chunked into 1-second windows.
 - Features extracted per window: mean, standard deviation, min, max, and magnitude across 6 axes.

2. Stress Classification (Rule-Based Engine)

Stress classification is calculated in the cloud via a deterministic Rule-Based Engine (rule-v1), NOT machine learning. This provides maximum explainability so owners know EXACTLY why their dog is stressed.

The algorithm uses a point-based scoring system based on real-time telemetry versus the dog's personalized baseline.

Rule	Condition	Points
Heart Rate (Elevated)	1.15x to 1.35x over baseline	+1
Heart Rate (Elevated)	1.35x to 1.6x over baseline	+2
Heart Rate (Elevated)	Over 1.6x over baseline	+3
Respiratory Rate (Elevated)	1.3x to 1.8x over baseline	+1
Respiratory Rate (Elevated)	Over 1.8x over baseline	+2
Motion (Restlessness)	Motion score 0.6 to 0.8	+1
Motion (Restlessness)	Motion score > 0.8	+2
Posture Combo	Moving + High Motion (>0.6)	+1
Environmental Amp.	Temp > 32C OR Humidity > 80%	+1
Rising Trend	Rising over 3 consecutive readings	+1

Final Stress Level	Score Range
Calm	0 - 1 points
Mild	2 - 3 points
Moderate	4 - 6 points
High	7+ points

PART 2: Rule-Based vs. Machine Learning

Why did FurFeel start with a Rule Engine for Stress?

1. The Cold Start Problem: You can't train an AI without thousands of labeled 'stressed' events. Rules allowed the app to launch immediately based on veterinary textbook thresholds.
2. Explainability: Rules output exact reasons ('HR is 1.6x baseline'), which is critical for medical liability and Care Insights.

Why you SHOULD transition to a Random Forest:

FurFeel - AI Architecture & Transition Roadmap

1. Non-Linear Patterns: Rules miss subtle combinations (e.g., HR slightly up + RR fluttering + micro-tremors). ML thrives on finding these invisible patterns.
2. Scalability: As you add more sensors (skin temp, acoustic barking), manual rules become impossible to maintain.
3. Continuous Learning: A Random Forest gets mathematically more accurate over time as users correct it.

PART 3: Roadmap to Random Forest Stress Classification

Phase 1: Data Collection & Ground Truth Labeling

An AI learns by looking at thousands of examples of the 'right answer.'

- Build a Tagging UI: Create a hidden developer/vet mode in the mobile app. When a dog is visibly stressed or calm, tap a button to log the Ground Truth.
- Sync to Supabase: Save these timestamps to a new 'clinical_stress_labels' table.
- Join the Data: Write a query that grabs the dog's telemetry for the 5 minutes prior to the button press, pairing it with the label (0=Calm, 1=Stressed).

Phase 2: Feature Engineering & Training (Python)

Once enough data is collected, write a Python script (e.g., ml/train_stress_model.py):

- Feature Engineering: Extract contextual features from raw data (e.g., hr_ratio, hr_rolling_variance, motion_sustained, temp_gradient).
- Train Model: Use Scikit-Learn to train a RandomForestClassifier(n_estimators=100, max_depth=10).
- Evaluate: Check the Confusion Matrix for accuracy, precision, and recall.

Phase 3: Deployment Architecture

Since the current engine runs in a Supabase Deno/TypeScript Edge Function, you have three deployment paths for the new AI model:

- Option A (ONNX / WebAssembly): Export the Python model to ONNX. Supabase Edge Functions support ONNX Runtime, allowing the AI to run in milliseconds entirely in TypeScript, without extra server costs.
- Option B (Microservice): Deploy a Python FastAPI server (e.g., on Fly.io/AWS). Supabase POSTs telemetry to this server, which runs the model and returns the score.
- Option C (Hardware): Export the model to C++ and run it directly on the collar firmware (like the posture model), saving cloud costs but requiring OTA firmware updates.

FurFeel - AI Architecture & Transition Roadmap

PART 4: Data Strategy for 4-Tier Classification

1. The Data Target

Because FurFeel tracks four distinct tiers (Calm, Mild, Moderate, High) instead of a simple binary (Calm vs. Stressed), the Random Forest requires sufficient data to mathematically define the boundaries for each specific bucket.

- Target: 300 to 500 labeled events per tier.
- Total Target: ~1,500 to 2,000 labeled events.
- Diversity Target: Data should be collected across 30 to 50 unique dogs of varying breeds, ages, and sizes to prevent overfitting.

2. The Subjectivity Problem (Mild vs. Moderate)

The hardest challenge in physiological machine learning is the subjectivity of the Ground Truth labels. If a dog is asleep, a human labeler easily tags 'Calm'. If a dog is violently shaking, they tag 'High'.

However, visually distinguishing between 'Mild' and 'Moderate' stress is highly subjective. If 10 different owners are asked to label a mildly anxious dog, they will provide conflicting labels. If the training data contains inconsistent guesses, the Random Forest will also learn to guess.

3. Industry Solutions for the 4-Tier Problem

To solve the blurriness of the middle tiers, the industry standard relies on one of two approaches:

Option A (Probabilities Interpolation):

A Random Forest Classifier is trained ONLY on the extremes (Calm vs. High Stress). Instead of outputting a definitive word, the model outputs a confidence percentage. A simple wrapper then translates this percentage into the 4 tiers (e.g., 30% confidence = Mild; 60% confidence = Moderate).

Option B (Random Forest Regressor):

Instead of predicting a discrete category, a Random Forest Regressor predicts a continuous physiological intensity score from 0.0 to 10.0. The app then applies strict threshold buckets to this continuous output, perfectly mimicking the current rule-based bucket system (e.g., 2.6 - 5.0 = Mild, 5.1 - 7.5 = Moderate).